

Звіт з лабораторної роботи № 3

Основи програмної інженерії

Дисципліна: Основи програмної інженерії

Тема: Модульне тестування програмного коду (Unit Testing)

Виконав: студент групи ПЗПІ-25-6 **Коновалов Олександр**

Email: oleksandr.konovalov1@nure.ua

Репозиторій: <https://github.com/oleksandrkonovalov1/bse-lr3-konovalov>

1. Тема та мета

Тема: Модульне тестування програмного коду

Мета: Набуття практичних навичок із написання модульних тестів з використанням фреймворку Vitest. Оволодіння техніками проектування тестів — еквівалентне розбиття (EP) та аналіз граничних значень (BVA). Досягнення line coverage $\geq 80\%$.

2. Вихідний код модуля з коментарями

Модуль реалізує систему створення академічних бібліографічних посилань на основі UML-моделі з ЛР 02. Мова: TypeScript. 12 класів, 3+ методи з нетривіальною логікою.

src/types.ts

```
/** Тип ідентифікатора публікації */
export enum IdentifierType {
  DOI = 'DOI',
  ISBN = 'ISBN',
  URL = 'URL',
}

/** Метадані публікації, отримані від провайдера */
export interface PublicationMetadata {
  title: string;
  authors: string[];
  year: number;
  publisher?: string;
  journal?: string;
  volume?: string;
  issue?: string;
  pages?: string;
  doi?: string;
  isbn?: string;
  url?: string;
}
```

src/search-request.ts

```
import { IdentifierType } from './types.js';

/** Запит на пошук публікації за ідентифікатором (DOI, ISBN або URL) */
export class SearchRequest {
  private readonly identifier: string;
  private readonly type: IdentifierType | null;

  constructor(identifier: string) {
    this.identifier = identifier.trim();
    this.type = this.detectType();
  }

  /** Перевіряє коректність ідентифікатора відповідно до його типу */
  validate(): boolean {
    if (!this.identifier || !this.type) return false;

    switch (this.type) {
      case IdentifierType.DOI:
        return /^10\.\d{4,}\.\S+$/.test(this.identifier);
      case IdentifierType.ISBN:
        return this.isValidIsbn();
      case IdentifierType.URL:
        try {
          const url = new URL(this.identifier);
          return url.protocol === 'http:' || url.protocol === 'https:';
        } catch {
          return false;
        }
    }
  }

  /** Повертає визначений тип ідентифікатора або кидає помилку, якщо тип не вдалось визначити */
  getType(): IdentifierType {
    if (!this.type) {
      throw new Error(`Cannot determine identifier type for: ${this.identifier}`);
    }
    return this.type;
  }

  /** Повертає нормалізований рядок ідентифікатора */
  getIdentifier(): string {
    return this.identifier;
  }

  private detectType(): IdentifierType | null {
    if (this.identifier.startsWith('10.')) return IdentifierType.DOI;
    if (/^https?:\/\/\//i.test(this.identifier)) return IdentifierType.URL;
    const cleaned = this.identifier.replace(/[-\s]/g, '');
  }
}
```

```

    if (/^\d{10}(\d{3})?$/).test(cleaned)) return IdentifierType.ISBN;
    return null;
}

private isValidIsbn(): boolean {
    const digits = this.identifier.replace(/[-\s]/g, '');
    if (digits.length === 10) return this.checksumIsbn10(digits);
    if (digits.length === 13) return this.checksumIsbn13(digits);
    return false;
}

private checksumIsbn10(digits: string): boolean {
    let sum = 0;
    for (let i = 0; i < 9; i++) {
        sum += (10 - i) * parseInt(digits[i]);
    }
    const last = digits[9].toUpperCase();
    sum += last === 'X' ? 10 : parseInt(last);
    return sum % 11 === 0;
}

private checksumIsbn13(digits: string): boolean {
    let sum = 0;
    for (let i = 0; i < 12; i++) {
        sum += (i % 2 === 0 ? 1 : 3) * parseInt(digits[i]);
    }
    const check = (10 - (sum % 10)) % 10;
    return check === parseInt(digits[12]);
}
}

```

src/citation-style.ts

```

import { PublicationMetadata } from './types.js';

export type StyleName = 'APA' | 'MLA' | 'Chicago';

const VALID_STYLES: StyleName[] = ['APA', 'MLA', 'Chicago'];

/** Стиль цитування (APA, MLA або Chicago) з логікою форматування метаданих */
export class CitationStyle {
    readonly name: StyleName;
    readonly template: string;

    constructor(name: StyleName) {
        if (!VALID_STYLES.includes(name)) {
            throw new Error(`Unknown citation style: ${name}`);
        }
        this.name = name;
        this.template = name;
    }
}

```

```

/** Форматує метадані публікації у рядок цитати відповідно до обраного стилю */
formatCitation(metadata: PublicationMetadata): string {
  switch (this.name) {
    case 'APA':
      return this.formatApa(metadata);
    case 'MLA':
      return this.formatMla(metadata);
    case 'Chicago':
      return this.formatChicago(metadata);
  }
}

private formatAuthorsApa(authors: string[]): string {
  if (authors.length === 0) return '';
  if (authors.length === 1) return `${authors[0]}.`;
  if (authors.length === 2) return `${authors[0]}, & ${authors[1]}.`;
  return `${authors[0]}, et al.`;
}

private formatApa(m: PublicationMetadata): string {
  const parts: string[] = [];

  const authorsStr = this.formatAuthorsApa(m.authors);
  if (authorsStr) parts.push(authorsStr);

  parts.push(`(${m.year}).`);

  if (m.journal) {
    parts.push(`${m.title}.`);
    let journalPart = `*${m.journal}*`;
    if (m.volume) journalPart += `, *${m.volume}*`;
    if (m.issue) journalPart += ` (${m.issue})`;
    if (m.pages) journalPart += `, ${m.pages}`;
    parts.push(`${journalPart}.`);
  } else {
    parts.push(`*${m.title}*.`);
  }

  if (m.publisher) parts.push(`${m.publisher}.`);
  if (m.doi) parts.push(`https://doi.org/${m.doi}`);

  return parts.join(' ');
}

private formatMla(m: PublicationMetadata): string {
  const parts: string[] = [];

  if (m.authors.length > 0) parts.push(`${m.authors[0]}.`);

  parts.push(`"${m.title}."`);

```

```

    if (m.journal) {
      let journalPart = `*${m.journal}*`;
      if (m.volume) journalPart += `, vol. ${m.volume}`;
      if (m.issue) journalPart += `, no. ${m.issue}`;
      journalPart += `, ${m.year}`;
      if (m.pages) journalPart += `, pp. ${m.pages}`;
      parts.push(`${journalPart}.`);
    } else if (m.publisher) {
      parts.push(`${m.publisher}, ${m.year}.`);
    }

    return parts.join(' ');
  }

  private formatChicago(m: PublicationMetadata): string {
    const parts: string[] = [];

    if (m.authors.length > 0) parts.push(`${m.authors[0]}.`);

    parts.push(`${m.title}.`);

    if (m.journal) {
      let journalPart = `*${m.journal}*`;
      if (m.volume) journalPart += ` ${m.volume}`;
      if (m.issue) journalPart += `, no. ${m.issue}`;
      journalPart += ` (${m.year})`;
      if (m.pages) journalPart += `: ${m.pages}`;
      parts.push(`${journalPart}.`);
    } else if (m.publisher) {
      parts.push(`${m.publisher}, ${m.year}.`);
    }

    if (m.doi) parts.push(`https://doi.org/${m.doi}`);

    return parts.join(' ');
  }
}

```

src/citation.ts

```

import { CitationStyle } from './citation-style.js';
import { PublicationMetadata } from './types.js';

let idCounter = 0;

/** Об'єкт цитати з метаданими, форматованим текстом та прив'язаним стилем */
export class Citation {
  readonly id: string;
  readonly rawMetadata: PublicationMetadata;
  private formattedText: string;
  private style: CitationStyle;
}

```

```

readonly createdAt: Date;

constructor(metadata: PublicationMetadata, style: CitationStyle) {
    this.id = `cit-${++idCounter}`;
    this.rawMetadata = metadata;
    this.style = style;
    this.formattedText = style.formatCitation(metadata);
    this.createdAt = new Date();
}

/** Переформатовує цитату у новому стилі та повертає оновлений текст */
format(style: CitationStyle): string {
    this.style = style;
    this.formattedText = style.formatCitation(this.rawMetadata);
    return this.formattedText;
}

/** Повертає поточний форматований текст цитати */
getFormattedText(): string {
    return this.formattedText;
}

/** Повертає поточний стиль цитування */
getStyle(): CitationStyle {
    return this.style;
}

/** Повертає форматований текст цитати для копіювання до буфера обміну */
copyToClipboard(): string {
    return this.formattedText;
}
}

```

src/citation-generator.ts

```

import { IdentifierType } from './types.js';
import { SearchRequest } from './search-request.js';
import { CitationStyle } from './citation-style.js';
import { Citation } from './citation.js';
import { MetadataProvider } from './metadata-provider.js';

/** Генератор цитат: валідація → маршрутизація до провайдера → форматування */
export class CitationGenerator {
    private readonly providers: Map<IdentifierType, MetadataProvider>;

    constructor(providers: Map<IdentifierType, MetadataProvider>) {
        if (providers.size === 0) {
            throw new Error('At least one metadata provider is required');
        }
        this.providers = providers;
    }
}

```

```

    /** Валідує запит, отримує метадані від відповідного провайдера та повертає
    відформатовану цитату */
    async generate(
        request: SearchRequest,
        style: CitationStyle,
    ): Promise<Citation> {
        if (!request.validate()) {
            throw new Error(
                `Invalid identifier: ${request.getIdentifier()}`,
            );
        }

        const provider = this.resolveProvider(request.getType());
        const metadata = await provider.fetchMetadata(request.getIdentifier());
        return new Citation(metadata, style);
    }

    /** Повертає провайдер метаданих для заданого типу ідентифікатора або кидає
    помилку */
    resolveProvider(type: IdentifierType): MetadataProvider {
        const provider = this.providers.get(type);
        if (!provider) {
            throw new Error(`No provider registered for type: ${type}`);
        }
        return provider;
    }
}

```

src/metadata-provider.ts

```

import { PublicationMetadata } from './types.js';

/** Інтерфейс провайдера метаданих публікації */
export interface MetadataProvider {
    /** Отримує метадані публікації за заданим ідентифікатором */
    fetchMetadata(identifier: string): Promise<PublicationMetadata>;
}

```

src/crossref-provider.ts

```

import { MetadataProvider } from './metadata-provider.js';
import { PublicationMetadata } from './types.js';

/** Провайдер метаданих для DOI через Crossref API */
export class CrossrefProvider implements MetadataProvider {
    private readonly apiUrl: string;

    constructor(apiUrl: string = 'https://api.crossref.org/works') {

```

```

    this.apiUrl = apiUrl;
  }

  async fetchMetadata(identifier: string): Promise<PublicationMetadata> {
    if (!identifier.startsWith('10.')) {
      throw new Error(`Invalid DOI format: ${identifier}`);
    }

    const response = await fetch(`${this.apiUrl}/${identifier}`);
    if (!response.ok) {
      throw new Error(`Crossref API error: ${response.status}`);
    }

    const data = await response.json();
    const item = data.message;

    return {
      title: item.title?.[0] ?? 'Unknown',
      authors: (item.author ?? []).map(
        (a: { family?: string; given?: string }) =>
          [a.family, a.given].filter(Boolean).join(', '),
      ),
      year: item.published?.['date-parts']?.[0]?.[0] ?? 0,
      journal: item['container-title']?.[0],
      volume: item.volume,
      issue: item.issue,
      pages: item.page,
      doi: identifier,
      publisher: item.publisher,
    };
  }
}

```

src/open-library-provider.ts

```

import { MetadataProvider } from './metadata-provider.js';
import { PublicationMetadata } from './types.js';

/** Провайдер метаданных для ISBN через Open Library API */
export class OpenLibraryProvider implements MetadataProvider {
  private readonly apiUrl: string;

  constructor(apiUrl: string = 'https://openlibrary.org/api/books') {
    this.apiUrl = apiUrl;
  }

  async fetchMetadata(identifier: string): Promise<PublicationMetadata> {
    const cleaned = identifier.replace(/[-\s]/g, '');
    if (!/^\d{10}(\d{3})?$/i.test(cleaned)) {
      throw new Error(`Invalid ISBN format: ${identifier}`);
    }
  }
}

```



```

const response = await fetch(
  `${this.apiUrl}?bibkeys=ISBN:${cleaned}&format=json&jscmd=data`,
);
if (!response.ok) {
  throw new Error(`Open Library API error: ${response.status}`);
}

const data = await response.json();
const book = data[`ISBN:${cleaned}`];
if (!book) {
  throw new Error(`Book not found for ISBN: ${identifier}`);
}

return {
  title: book.title ?? 'Unknown',
  authors: (book.authors ?? []).map((a: { name: string }) => a.name),
  year: book.publish_date ? parseInt(book.publish_date, 10) : 0,
  publisher: book.publishers?.[0]?.name,
  isbn: identifier,
};
}
}

```

src/web-scraper-provider.ts

```

import { MetadataProvider } from './metadata-provider.js';
import { PublicationMetadata } from './types.js';

/** Провайдер метаданих для URL через HTTP-запит і парсинг HTML-заголовку */
export class WebScraperProvider implements MetadataProvider {
  async fetchMetadata(url: string): Promise<PublicationMetadata> {
    try {
      new URL(url);
    } catch {
      throw new Error(`Invalid URL: ${url}`);
    }

    const response = await fetch(url);
    if (!response.ok) {
      throw new Error(`Failed to fetch URL: ${response.status}`);
    }

    const html = await response.text();
    const title = this.extractTitle(html);

    return {
      title,
      authors: [],
      year: new Date().getFullYear(),
      url,
    };
  }
}

```

```

    };
  }

  private extractTitle(html: string): string {
    const match = html.match(/<title[^>]*>([^\<]+)<\/title>/i);
    return match ? match[1].trim() : 'Unknown';
  }
}

```

src/user.ts

```

/** Абстрактний базовий клас користувача системи цитування */
export abstract class User {
  protected readonly id: string;
  protected readonly email: string;

  constructor(id: string, email: string) {
    this.id = id;
    this.email = email;
  }

  /** Повертає роль користувача (guest, registered або admin) */
  abstract getRole(): string;

  /** Повертає унікальний ідентифікатор користувача */
  getId(): string {
    return this.id;
  }

  /** Повертає email-адресу користувача */
  getEmail(): string {
    return this.email;
  }
}

```

src/guest.ts

```

import { User } from './user.js';
import { SearchRequest } from './search-request.js';
import { CitationStyle } from './citation-style.js';
import { Citation } from './citation.js';
import { CitationGenerator } from './citation-generator.js';

/** Гостьовий користувач – може генерувати цитати без реєстрації */
export class Guest extends User {
  private readonly generator: CitationGenerator;

  constructor(generator: CitationGenerator) {
    super(`guest-${Date.now()}`, '');
  }
}

```

```

    this.generator = generator;
}

/** Повертає роль "guest" */
getRole(): string {
    return 'guest';
}

/** Генерує цитату за запитом та стилем через спільний генератор */
async generateCitation(
    request: SearchRequest,
    style: CitationStyle,
): Promise<Citation> {
    return this.generator.generate(request, style);
}
}

```

src/registered-user.ts

```

import { User } from './user.js';
import { SearchRequest } from './search-request.js';
import { CitationStyle } from './citation-style.js';
import { Citation } from './citation.js';
import { CitationGenerator } from './citation-generator.js';

/** Зареєстрований користувач із можливістю зберігати та переглядати історію цитат */
export class RegisteredUser extends User {
    private savedCitations: Citation[] = [];
    private readonly generator: CitationGenerator;

    constructor(id: string, email: string, generator: CitationGenerator) {
        super(id, email);
        this.generator = generator;
    }

    /** Повертає роль "registered" */
    getRole(): string {
        return 'registered';
    }

    /** Генерує цитату за запитом та стилем через спільний генератор */
    async generateCitation(
        request: SearchRequest,
        style: CitationStyle,
    ): Promise<Citation> {
        return this.generator.generate(request, style);
    }

    /** Зберігає цитату до особистої бібліотеки; кидає помилку при дублікаті */
    saveCitation(citation: Citation): void {

```

```

        if (this.savedCitations.some((c) => c.id === citation.id)) {
            throw new Error(`Citation ${citation.id} already saved`);
        }
        this.savedCitations.push(citation);
    }

    /** Повертає копію списку збережених цитат користувача */
    getHistory(): Citation[] {
        return [...this.savedCitations];
    }

    /** Перевіряє облікові дані користувача: email повинен збігатися, пароль – мінімум 8 символів */
    login(email: string, password: string): boolean {
        if (!email || !password) return false;
        return this.email === email && password.length >= 8;
    }
}

```

src/admin.ts

```

import { User } from './user.js';
import { CitationStyle, StyleName } from './citation-style.js';

/** Адміністратор системи – керує користувачами та стилями цитування */
export class Admin extends User {
    private readonly users: Map<string, User> = new Map();
    private readonly styles: Map<string, CitationStyle> = new Map();

    constructor(id: string, email: string) {
        super(id, email);
    }

    /** Повертає роль "admin" */
    getRole(): string {
        return 'admin';
    }

    /** Додає користувача до реєстру адміністратора */
    addUser(user: User): void {
        this.users.set(user.getId(), user);
    }

    /** Повертає список усіх зареєстрованих у реєстрі користувачів */
    manageUsers(): User[] {
        return [...this.users.values()];
    }

    /** Видаляє користувача з реєстру за ідентифікатором; повертає true якщо видалення успішне */
    deleteUser(userId: string): boolean {

```

```
        return this.users.delete(userId);
    }

    /** Додає новий стиль цитування до реєстру */
    addStyle(name: StyleName): void {
        this.styles.set(name, new CitationStyle(name));
    }

    /** Повертає список усіх доступних стилів цитування */
    manageStyles(): CitationStyle[] {
        return [...this.styles.values()];
    }

    /** Перевіряє облікові дані адміністратора: email повинен збігатися, пароль –
    мінімум 8 символів */
    login(email: string, password: string): boolean {
        if (!email || !password) return false;
        return this.email === email && password.length >= 8;
    }
}
```

3. Таблиця проектування тестів

SearchRequest.validate()

Тест-кейс	Вхідні дані	Очікуваний результат	Техніка	Статус
Валідний DOI	10.1000/xyz	true	EP	pass
DOI зі складним суфіксом	10.1038/s41586-021-03819-2	true	EP	pass
DOI з коротким реєстрантом	10.1/x	false	BVA	pass
Мінімальний валідний DOI	10.1000/x	true	BVA	pass
Валідний ISBN-13 з дефісами	978-3-16-148410-0	true	EP	pass
Валідний ISBN-10	0306406152	true	EP	pass
ISBN-13 з невірною контрольною сумою	978-3-16-148410-1	false	EP	pass
Валідний HTTPS URL	https://example.com/article	true	EP	pass
Валідний HTTP URL	http://example.com	true	EP	pass
Порожній рядок	""	false	BVA	pass

Рядок з пробілів	" "	false	BVA	pass
------------------	-----	-------	-----	------

SearchRequest.getType()

Тест-кейс	Вхідні дані	Очікуваний результат	Техніка	Статус
Визначення DOI	10.1000/xyz	IdentifierType.DOI	EP	pass
Визначення URL	https://example.com	IdentifierType.URL	EP	pass
Визначення ISBN	0306406152	IdentifierType.ISBN	EP	pass
Нерозпізнаний ідентифікатор	foobar	throws Error	EP	pass

CitationStyle.formatCitation()

Тест-кейс	Вхідні дані	Очікуваний результат	Техніка	Статус
APA: журнальна стаття, 2 автори	journal + 2 authors	Формат APA з &	EP	pass
APA: книга, 1 автор	book + 1 author	Курсив назви, видавець	EP	pass
APA: без авторів	0 authors	Без префіксу автора	BVA	pass
APA: 3+ авторів	3 authors	et al.	BVA	pass
MLA: журнальна стаття	journal article	vol., no.	EP	pass
MLA: книга	book + publisher	Видавець, рік	EP	pass
Chicago: стаття з DOI	journal + DOI	doi.org посилання	EP	pass
Chicago: книга	book + publisher	Видавець, рік	EP	pass
Chicago: мінімальні дані	no DOI	Без DOI посилання	EP	pass

CitationGenerator

Тест-кейс	Вхідні дані	Очікуваний результат	Техніка	Статус
Порожня мапа провайдерів	new Map()	throws Error	BVA	pass
Зареєстрований тип	DOI type	Повертає провайдер	EP	pass
Незареєстрований тип	ISBN (не зареєстрований)	throws Error	EP	pass

Генерація для валідного запиту	DOI + APA	Citation об'єкт	Positive	pass
Невалідний ідентифікатор	invalid	throws Error	Negative	pass
Помилка провайдера	mock reject	throws Error	Negative	pass

Providers (CrossrefProvider, OpenLibraryProvider, WebScraperProvider)

Тест-кейс	Вхідні дані	Очікуваний результат	Техніка	Статус
Crossref: успішний запит	mock API response	PublicationMetadata	Positive	pass
Crossref: невалідний DOI	invalid	throws Error	EP	pass
Crossref: помилка API	404 response	throws Error	Negative	pass
Crossref: неповні дані	empty message	defaults (Unknown, [], 0)	EP	pass
OpenLibrary: успішний запит	mock API response	PublicationMetadata	Positive	pass
OpenLibrary: невалідний ISBN	abc	throws Error	EP	pass
OpenLibrary: книга не знайдена	empty response	throws Error	Negative	pass
OpenLibrary: помилка API	500 response	throws Error	Negative	pass
WebScraper: витягує title	HTML з <title>	Title text	Positive	pass
WebScraper: без title тегу	HTML без <title>	Unknown	EP	pass
WebScraper: невалідний URL	not-a-url	throws Error	EP	pass
WebScraper: помилка HTTP	403 response	throws Error	Negative	pass

User Hierarchy (Guest, RegisteredUser, Admin)

Тест-кейс	Вхідні дані	Очікуваний результат	Техніка	Статус
Guest: роль	—	guest	Positive	pass
Guest: автогенерований id	—	matches /^guest-/	Positive	pass
RegisteredUser: роль	—	registered	Positive	pass
Login: вірні дані	correct email + 8+ chars	true	EP	pass

Login: невірний email	wrong email	false	EP	pass
Login: пароль 7 символів	7-char password	false	BVA	pass
Login: пароль 8 символів	8-char password	true	BVA	pass
Login: порожній email	empty email	false	EP	pass
Збереження цитати	Citation object	history length = 1	Positive	pass
Дублікат цитати	same Citation twice	throws Error	Negative	pass
Порожня історія	—	[]	BVA	pass
Admin: роль	—	admin	Positive	pass
Admin: додавання користувача	Guest object	manageUsers length = 1	Positive	pass
Admin: видалення користувача	existing userId	true, length = 0	Positive	pass
Admin: видалення неіснуючого	unknown userId	false	Negative	pass
Admin: додавання стилів	APA + MLA	manageStyles length = 2	Positive	pass
Admin: login	correct credentials	true	EP	pass

4. Вихідний код тестового набору з коментарями

Тести написані з використанням фреймворку **Vitest**. Кожен тест відповідає патерну **AAA** (Arrange — Act — Assert) та містить коментар із зазначенням техніки (EP/BVA/Positive/Negative).

src/search-request.test.ts

```
import { describe, it, expect } from 'vitest';
import { SearchRequest } from './search-request.js';
import { IdentifierType } from './types.js';

describe('SearchRequest', () => {
  describe('validate()', () => {
    it('accepts valid DOI', () => { // EP: valid DOI class
      // Arrange
      const req = new SearchRequest('10.1000/xyz');
      // Act
      const result = req.validate();
      // Assert
      expect(result).toBe(true);
    });

    it('accepts DOI with complex suffix', () => { // EP: valid DOI, complex suffix
```



```

    // Arrange
    const req = new SearchRequest('10.1038/s41586-021-03819-2');
    // Act & Assert
    expect(req.validate()).toBe(true);
  });

  it('rejects DOI with registrant shorter than 4 digits', () => { // BVA: DOI
    registrant boundary
    // Arrange
    const req = new SearchRequest('10.1/x');
    // Act & Assert
    expect(req.validate()).toBe(false);
  });

  it('accepts DOI with exactly 4-digit registrant', () => { // BVA: minimal valid
    DOI
    // Arrange
    const req = new SearchRequest('10.1000/x');
    // Act & Assert
    expect(req.validate()).toBe(true);
  });

  it('accepts valid ISBN-13 with hyphens', () => { // EP: valid ISBN-13
    // Arrange
    const req = new SearchRequest('978-3-16-148410-0');
    // Act & Assert
    expect(req.validate()).toBe(true);
  });

  it('accepts valid ISBN-10', () => { // EP: valid ISBN-10
    // Arrange
    const req = new SearchRequest('0306406152');
    // Act & Assert
    expect(req.validate()).toBe(true);
  });

  it('rejects ISBN-13 with bad checksum', () => { // EP: invalid ISBN checksum
    // Arrange
    const req = new SearchRequest('978-3-16-148410-1');
    // Act & Assert
    expect(req.validate()).toBe(false);
  });

  it('accepts valid HTTPS URL', () => { // EP: valid URL
    // Arrange
    const req = new SearchRequest('https://example.com/article');
    // Act & Assert
    expect(req.validate()).toBe(true);
  });

  it('accepts valid HTTP URL', () => { // EP: valid URL, http protocol
    // Arrange

```

```

    const req = new SearchRequest('http://example.com');
    // Act & Assert
    expect(req.validate()).toBe(true);
  });

  it('returns false for empty string', () => { // BVA: empty input boundary
    // Arrange
    const req = new SearchRequest('');
    // Act & Assert
    expect(req.validate()).toBe(false);
  });

  it('returns false for whitespace-only string', () => { // BVA: whitespace
boundary
    // Arrange
    const req = new SearchRequest(' ');
    // Act & Assert
    expect(req.validate()).toBe(false);
  });
});

describe('getType()', () => {
  it('detects DOI type', () => { // EP: DOI detection
    // Arrange & Act
    const req = new SearchRequest('10.1000/xyz');
    // Assert
    expect(req.getType()).toBe(IdentifierType.DOI);
  });

  it('detects URL type', () => { // EP: URL detection
    // Arrange & Act
    const req = new SearchRequest('https://example.com');
    // Assert
    expect(req.getType()).toBe(IdentifierType.URL);
  });

  it('detects ISBN type', () => { // EP: ISBN detection
    // Arrange & Act
    const req = new SearchRequest('0306406152');
    // Assert
    expect(req.getType()).toBe(IdentifierType.ISBN);
  });

  it('throws for unrecognizable identifier', () => { // EP: unrecognizable input
    // Arrange
    const req = new SearchRequest('foobar');
    // Act & Assert
    expect(() => req.getType()).toThrow('Cannot determine identifier type');
  });
});
});

```

src/citation-style.test.ts

```
import { describe, it, expect } from 'vitest';
import { CitationStyle } from './citation-style.js';
import { PublicationMetadata } from './types.js';

const journalArticle: PublicationMetadata = {
  title: 'Deep Learning',
  authors: ['Smith, J.', 'Jones, A.'],
  year: 2023,
  journal: 'Nature',
  volume: '521',
  issue: '7553',
  pages: '436-444',
  doi: '10.1038/nature14539',
};

const book: PublicationMetadata = {
  title: 'Clean Code',
  authors: ['Martin, R.'],
  year: 2008,
  publisher: 'Prentice Hall',
};

const minimalMetadata: PublicationMetadata = {
  title: 'Untitled Work',
  authors: [],
  year: 2024,
};

describe('CitationStyle', () => {
  describe('APA formatting', () => {
    const apa = new CitationStyle('APA');

    it('formats journal article with two authors', () => { // EP: journal article +
2 authors
      // Arrange – uses journalArticle fixture
      // Act
      const result = apa.formatCitation(journalArticle);
      // Assert
      expect(result).toContain('Smith, J., & Jones, A. ');
      expect(result).toContain('(2023). ');
      expect(result).toContain('Deep Learning. ');
      expect(result).toContain('*Nature*');
      expect(result).toContain('https://doi.org/10.1038/nature14539');
    });

    it('formats book with single author', () => { // EP: book (no journal) + 1
author
      // Arrange – uses book fixture
      // Act
```

```

    const result = apa.formatCitation(book);
    // Assert
    expect(result).toContain('Martin, R. ');
    expect(result).toContain('*Clean Code*');
    expect(result).toContain('Prentice Hall. ');
  });

  it('formats with no authors', () => { // BVA: 0 authors (boundary)
    // Arrange – uses minimalMetadata fixture
    // Act
    const result = apa.formatCitation(minimalMetadata);
    // Assert
    expect(result).toContain('(2024). ');
    expect(result).toContain('*Untitled Work*');
    expect(result).not.toMatch(/^\s*\./);
  });

  it('uses et al. for 3+ authors', () => { // BVA: 3 authors (boundary for et al.)
    // Arrange
    const meta: PublicationMetadata = {
      title: 'Study',
      authors: ['A', 'B', 'C'],
      year: 2020,
    };
    // Act
    const result = apa.formatCitation(meta);
    // Assert
    expect(result).toContain('A, et al. ');
  });
});

describe('MLA formatting', () => {
  const mla = new CitationStyle('MLA');

  it('formats journal article', () => { // EP: MLA journal
    // Act
    const result = mla.formatCitation(journalArticle);
    // Assert
    expect(result).toContain('Smith, J. ');
    expect(result).toContain('"Deep Learning."');
    expect(result).toContain('*Nature*');
    expect(result).toContain('vol. 521');
    expect(result).toContain('no. 7553');
  });

  it('formats book with publisher', () => { // EP: MLA book
    // Act
    const result = mla.formatCitation(book);
    // Assert
    expect(result).toContain('"Clean Code."');
    expect(result).toContain('Prentice Hall, 2008. ');
  });
});

```

```

});

describe('Chicago formatting', () => {
  const chicago = new CitationStyle('Chicago');

  it('formats journal article with DOI', () => { // EP: Chicago journal
    // Act
    const result = chicago.formatCitation(journalArticle);
    // Assert
    expect(result).toContain('"Deep Learning."');
    expect(result).toContain('*Nature* 521');
    expect(result).toContain('(2023)');
    expect(result).toContain('https://doi.org/');
  });

  it('formats book with publisher', () => { // EP: Chicago book
    // Act
    const result = chicago.formatCitation(book);
    // Assert
    expect(result).toContain('"Clean Code."');
    expect(result).toContain('Prentice Hall, 2008. ');
  });

  it('formats minimal metadata without DOI', () => { // EP: Chicago minimal
    // Act
    const result = chicago.formatCitation(minimalMetadata);
    // Assert
    expect(result).not.toContain('doi.org');
  });
});
});

```

src/citation.test.ts

```

import { describe, it, expect } from 'vitest';
import { Citation } from './citation.js';
import { CitationStyle } from './citation-style.js';
import { PublicationMetadata } from './types.js';

const metadata: PublicationMetadata = {
  title: 'Test Article',
  authors: ['Author, A.'],
  year: 2024,
  journal: 'Journal',
};

describe('Citation', () => {
  it('creates citation with formatted text', () => { // Positive: constructor
    // Arrange
    const style = new CitationStyle('APA');
    // Act

```

```

    const citation = new Citation(metadata, style);
    // Assert
    expect(citation.getFormattedText()).toContain('Test Article');
    expect(citation.getStyle().name).toBe('APA');
    expect(citation.id).toBeDefined();
    expect(citation.createdAt).toBeInstanceOf(Date);
  });

  it('re-formats citation with different style', () => { // Positive: format()
    // Arrange
    const citation = new Citation(metadata, new CitationStyle('APA'));
    // Act
    const result = citation.format(new CitationStyle('MLA'));
    // Assert
    expect(result).toContain('"Test Article."');
    expect(citation.getStyle().name).toBe('MLA');
  });

  it('copyToClipboard returns formatted text', () => { // Positive:
copyToClipboard()
    // Arrange
    const citation = new Citation(metadata, new CitationStyle('APA'));
    // Act
    const text = citation.copyToClipboard();
    // Assert
    expect(text).toBe(citation.getFormattedText());
  });

  it('stores raw metadata unchanged', () => { // Positive: rawMetadata preservation
    // Arrange & Act
    const citation = new Citation(metadata, new CitationStyle('APA'));
    // Assert
    expect(citation.rawMetadata).toEqual(metadata);
  });
});

```

src/citation-generator.test.ts

```

import { describe, it, expect, vi } from 'vitest';
import { CitationGenerator } from './citation-generator.js';
import { SearchRequest } from './search-request.js';
import { CitationStyle } from './citation-style.js';
import { IdentifierType } from './types.js';
import type { MetadataProvider } from './metadata-provider.js';

function mockProvider(): MetadataProvider {
  return {
    fetchMetadata: vi.fn().mockResolvedValue({
      title: 'Mock Article',
      authors: ['Mock Author'],
      year: 2024,

```

```

    }},
  };
}

describe('CitationGenerator', () => {
  it('throws when created with empty providers map', () => { // BVA: empty providers
    boundary
    // Arrange & Act & Assert
    expect(() => new CitationGenerator(new Map())).toThrow('At least one metadata
    provider is required');
  });

  describe('resolveProvider()', () => {
    it('returns provider for registered type', () => { // EP: registered type
      // Arrange
      const doiProvider = mockProvider();
      const gen = new CitationGenerator(new Map([[IdentifierType.DOI,
    doiProvider]]));
      // Act
      const result = gen.resolveProvider(IdentifierType.DOI);
      // Assert
      expect(result).toBe(doiProvider);
    });

    it('throws for unregistered type', () => { // EP: unregistered type
      // Arrange
      const gen = new CitationGenerator(new Map([[IdentifierType.DOI,
    mockProvider()]]));
      // Act & Assert
      expect(() => gen.resolveProvider(IdentifierType.ISBN)).toThrow('No provider
    registered for type: ISBN');
    });
  });

  describe('generate()', () => {
    it('generates citation for valid request', async () => { // Positive: full flow
      // Arrange
      const provider = mockProvider();
      const gen = new CitationGenerator(new Map([[IdentifierType.DOI, provider]]));
      const request = new SearchRequest('10.1000/test');
      const style = new CitationStyle('APA');
      // Act
      const citation = await gen.generate(request, style);
      // Assert
      expect(citation.rawMetadata.title).toBe('Mock Article');
      expect(citation.getStyle().name).toBe('APA');
      expect(provider.fetchMetadata).toHaveBeenCalledWith('10.1000/test');
    });

    it('throws for invalid identifier', async () => { // Negative: validation
    failure
      // Arrange

```

```

    const gen = new CitationGenerator(new Map([[IdentifierType.DOI,
mockProvider()]]));
    const request = new SearchRequest('invalid');
    const style = new CitationStyle('APA');
    // Act & Assert
    await expect(gen.generate(request, style)).rejects.toThrow('Invalid
identifier');
  });

  it('propagates provider errors', async () => { // Negative: provider failure
    // Arrange
    const failingProvider: MetadataProvider = {
      fetchMetadata: vi.fn().mockRejectedValue(new Error('API down')),
    };
    const gen = new CitationGenerator(new Map([[IdentifierType.DOI,
failingProvider]]));
    const request = new SearchRequest('10.1000/test');
    const style = new CitationStyle('APA');
    // Act & Assert
    await expect(gen.generate(request, style)).rejects.toThrow('API down');
  });
});
});
});

```

src/crossref-provider.test.ts

```

import { describe, it, expect, vi, beforeEach } from 'vitest';
import { CrossrefProvider } from './crossref-provider.js';

describe('CrossrefProvider', () => {
  beforeEach(() => {
    vi.restoreAllMocks();
  });

  it('parses Crossref API response into metadata', async () => { // Positive:
successful fetch
    // Arrange
    const provider = new CrossrefProvider();
    vi.stubGlobal('fetch', vi.fn().mockResolvedValue({
      ok: true,
      json: async () => ({
        message: {
          title: ['Test Article'],
          author: [{ family: 'Smith', given: 'John' }],
          published: { 'date-parts': [[2023]] },
          'container-title': ['Nature'],
          volume: '1',
          publisher: 'Springer',
        },
      })),
    }));
  });
});

```



```

// Act
const metadata = await provider.fetchMetadata('10.1000/test');
// Assert
expect(metadata.title).toBe('Test Article');
expect(metadata.authors).toEqual(['Smith, John']);
expect(metadata.year).toBe(2023);
expect(metadata.journal).toBe('Nature');
});

it('throws on invalid DOI format', async () => { // EP: invalid DOI
// Arrange
const provider = new CrossrefProvider();
// Act & Assert
await expect(provider.fetchMetadata('invalid')).rejects.toThrow('Invalid DOI
format');
});

it('throws on API error response', async () => { // Negative: API failure
// Arrange
const provider = new CrossrefProvider();
vi.stubGlobal('fetch', vi.fn().mockResolvedValue({ ok: false, status: 404 }));
// Act & Assert
await expect(provider.fetchMetadata('10.1000/test')).rejects.toThrow('Crossref
API error: 404');
});

it('handles missing fields gracefully', async () => { // EP: sparse API response
// Arrange
const provider = new CrossrefProvider();
vi.stubGlobal('fetch', vi.fn().mockResolvedValue({
  ok: true,
  json: async () => ({ message: {} }),
}));
// Act
const metadata = await provider.fetchMetadata('10.1000/test');
// Assert
expect(metadata.title).toBe('Unknown');
expect(metadata.authors).toEqual([]);
expect(metadata.year).toBe(0);
});
});

```

src/open-library-provider.test.ts

```

import { describe, it, expect, vi, beforeEach } from 'vitest';
import { OpenLibraryProvider } from './open-library-provider.js';

describe('OpenLibraryProvider', () => {
  beforeEach(() => {
    vi.restoreAllMocks();
  });
});

```

```

    it('parses Open Library response into metadata', async () => { // Positive:
successful fetch
    // Arrange
    const provider = new OpenLibraryProvider();
    vi.stubGlobal('fetch', vi.fn().mockResolvedValue({
      ok: true,
      json: async () => ({
        'ISBN:0306406152': {
          title: 'Test Book',
          authors: [{ name: 'Author A' }],
          publish_date: '2020',
          publishers: [{ name: 'Publisher X' }],
        },
      })),
    });
    // Act
    const metadata = await provider.fetchMetadata('0306406152');
    // Assert
    expect(metadata.title).toBe('Test Book');
    expect(metadata.authors).toEqual(['Author A']);
    expect(metadata.year).toBe(2020);
  });

  it('throws on invalid ISBN format', async () => { // EP: invalid ISBN
    // Arrange
    const provider = new OpenLibraryProvider();
    // Act & Assert
    await expect(provider.fetchMetadata('abc')).rejects.toThrow('Invalid ISBN
format');
  });

  it('throws when book not found', async () => { // Negative: book not found
    // Arrange
    const provider = new OpenLibraryProvider();
    vi.stubGlobal('fetch', vi.fn().mockResolvedValue({
      ok: true,
      json: async () => ({}),
    }));
    // Act & Assert
    await expect(provider.fetchMetadata('0306406152')).rejects.toThrow('Book not
found');
  });

  it('throws on API error', async () => { // Negative: API failure
    // Arrange
    const provider = new OpenLibraryProvider();
    vi.stubGlobal('fetch', vi.fn().mockResolvedValue({ ok: false, status: 500 }));
    // Act & Assert
    await expect(provider.fetchMetadata('0306406152')).rejects.toThrow('Open Library
API error: 500');
  });

```

```
});  
});
```

src/web-scraper-provider.test.ts

```
import { describe, it, expect, vi, beforeEach } from 'vitest';  
import { WebScraperProvider } from './web-scraper-provider.js';  
  
describe('WebScraperProvider', () => {  
  beforeEach(() => {  
    vi.restoreAllMocks();  
  });  
  
  it('extracts title from HTML page', async () => { // Positive: title extraction  
    // Arrange  
    const provider = new WebScraperProvider();  
    vi.stubGlobal('fetch', vi.fn().mockResolvedValue({  
      ok: true,  
      text: async () => '<html><title>Page Title</title><body></body></html>',  
    }));  
    // Act  
    const metadata = await provider.fetchMetadata('https://example.com');  
    // Assert  
    expect(metadata.title).toBe('Page Title');  
    expect(metadata.url).toBe('https://example.com');  
  });  
  
  it('returns "Unknown" when no title tag', async () => { // EP: missing title  
    // Arrange  
    const provider = new WebScraperProvider();  
    vi.stubGlobal('fetch', vi.fn().mockResolvedValue({  
      ok: true,  
      text: async () => '<html><body>No title here</body></html>',  
    }));  
    // Act  
    const metadata = await provider.fetchMetadata('https://example.com');  
    // Assert  
    expect(metadata.title).toBe('Unknown');  
  });  
  
  it('throws on invalid URL', async () => { // EP: invalid URL  
    // Arrange  
    const provider = new WebScraperProvider();  
    // Act & Assert  
    await expect(provider.fetchMetadata('not-a-url')).rejects.toThrow('Invalid  
URL');  
  });  
  
  it('throws on fetch failure', async () => { // Negative: HTTP error  
    // Arrange  
    const provider = new WebScraperProvider();
```

```

    vi.stubGlobal('fetch', vi.fn().mockResolvedValue({ ok: false, status: 403 }));
    // Act & Assert
    await
    expect(provider.fetchMetadata('https://example.com')).rejects.toThrow('Failed to
    fetch URL: 403');
  });
});

```

src/user.test.ts

```

import { describe, it, expect, vi } from 'vitest';
import { Guest } from './guest.js';
import { RegisteredUser } from './registered-user.js';
import { Admin } from './admin.js';
import { CitationGenerator } from './citation-generator.js';
import { Citation } from './citation.js';
import { CitationStyle } from './citation-style.js';
import { IdentifierType } from './types.js';
import type { MetadataProvider } from './metadata-provider.js';

function createGenerator(): CitationGenerator {
  const provider: MetadataProvider = {
    fetchMetadata: vi.fn().mockResolvedValue({
      title: 'Test', authors: ['A'], year: 2024,
    }),
  };
  return new CitationGenerator(new Map([[IdentifierType.DOI, provider]]));
}

describe('Guest', () => {
  it('returns "guest" role', () => { // Positive: role check
    // Arrange & Act
    const guest = new Guest(createGenerator());
    // Assert
    expect(guest.getRole()).toBe('guest');
  });

  it('has auto-generated id and empty email', () => { // Positive: constructor
    // Arrange & Act
    const guest = new Guest(createGenerator());
    // Assert
    expect(guest.getId()).toMatch(/^guest-/);
    expect(guest.getEmail()).toBe('');
  });
});

describe('RegisteredUser', () => {
  it('returns "registered" role', () => { // Positive: role check
    // Arrange & Act
    const user = new RegisteredUser('u1', 'user@test.com', createGenerator());
    // Assert
  });
});

```

```

    expect(user.getRole()).toBe('registered');
  });

it('login succeeds with correct credentials', () => { // EP: valid login
  // Arrange
  const user = new RegisteredUser('u1', 'user@test.com', createGenerator());
  // Act & Assert
  expect(user.login('user@test.com', 'password123')).toBe(true);
});

it('login fails with wrong email', () => { // EP: wrong email
  // Arrange
  const user = new RegisteredUser('u1', 'user@test.com', createGenerator());
  // Act & Assert
  expect(user.login('wrong@test.com', 'password123')).toBe(false);
});

it('login fails with short password', () => { // BVA: password length boundary (7
chars)
  // Arrange
  const user = new RegisteredUser('u1', 'user@test.com', createGenerator());
  // Act & Assert
  expect(user.login('user@test.com', '1234567')).toBe(false);
});

it('login succeeds with exactly 8-char password', () => { // BVA: password length
boundary (8 chars)
  // Arrange
  const user = new RegisteredUser('u1', 'user@test.com', createGenerator());
  // Act & Assert
  expect(user.login('user@test.com', '12345678')).toBe(true);
});

it('login fails with empty email', () => { // EP: empty credentials
  // Arrange
  const user = new RegisteredUser('u1', 'user@test.com', createGenerator());
  // Act & Assert
  expect(user.login('', 'password123')).toBe(false);
});

it('saves citation and retrieves history', () => { // Positive: save + history
  // Arrange
  const user = new RegisteredUser('u1', 'user@test.com', createGenerator());
  const citation = new Citation(
    { title: 'T', authors: [], year: 2024 },
    new CitationStyle('APA'),
  );
  // Act
  user.saveCitation(citation);
  // Assert
  expect(user.getHistory()).toHaveLength(1);
  expect(user.getHistory()[0].id).toBe(citation.id);

```

```

});

it('throws when saving duplicate citation', () => { // Negative: duplicate save
  // Arrange
  const user = new RegisteredUser('u1', 'user@test.com', createGenerator());
  const citation = new Citation(
    { title: 'T', authors: [], year: 2024 },
    new CitationStyle('APA'),
  );
  user.saveCitation(citation);
  // Act & Assert
  expect(() => user.saveCitation(citation)).toThrow('already saved');
});

it('returns empty history initially', () => { // BVA: empty history boundary
  // Arrange
  const user = new RegisteredUser('u1', 'user@test.com', createGenerator());
  // Act & Assert
  expect(user.getHistory()).toEqual([]);
});
});

describe('Admin', () => {
  it('returns "admin" role', () => { // Positive: role check
    // Arrange & Act
    const admin = new Admin('a1', 'admin@test.com');
    // Assert
    expect(admin.getRole()).toBe('admin');
  });

  it('adds and lists users', () => { // Positive: user management
    // Arrange
    const admin = new Admin('a1', 'admin@test.com');
    const guest = new Guest(createGenerator());
    // Act
    admin.addUser(guest);
    // Assert
    expect(admin.manageUsers()).toHaveLength(1);
  });

  it('deletes existing user', () => { // Positive: delete user
    // Arrange
    const admin = new Admin('a1', 'admin@test.com');
    const guest = new Guest(createGenerator());
    admin.addUser(guest);
    // Act
    const result = admin.deleteUser(guest.getId());
    // Assert
    expect(result).toBe(true);
    expect(admin.manageUsers()).toHaveLength(0);
  });
});

```

```

    it('returns false when deleting non-existent user', () => { // Negative: delete
unknown user
    // Arrange
    const admin = new Admin('a1', 'admin@test.com');
    // Act & Assert
    expect(admin.deleteUser('unknown-id')).toBe(false);
});

it('adds and lists styles', () => { // Positive: style management
    // Arrange
    const admin = new Admin('a1', 'admin@test.com');
    // Act
    admin.addStyle('APA');
    admin.addStyle('MLA');
    // Assert
    expect(admin.manageStyles()).toHaveLength(2);
});

it('login works with correct credentials', () => { // EP: valid admin login
    // Arrange
    const admin = new Admin('a1', 'admin@test.com');
    // Act & Assert
    expect(admin.login('admin@test.com', 'adminpass')).toBe(true);
});
});

```

5. Покриття коду

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	96.11	94.8	96.61	96.11	
admin.ts	100	90	100	100	45
...n-generator.ts	100	100	100	100	
citation-style.ts	97.5	97.22	100	97.5	14-15
citation.ts	100	100	100	100	
...ef-provider.ts	100	100	100	100	
guest.ts	70.58	100	75	70.58	23-27
...ta-provider.ts	0	0	0	0	
...ry-provider.ts	100	78.57	100	100	32-34
...stered-user.ts	83.87	100	85.71	83.87	24-28
search-request.ts	95.31	90.9	100	95.31	27-28,57
types.ts	100	100	100	100	
user.ts	100	100	100	100	
...er-provider.ts	100	100	100	100	

Line coverage: 96.11% (поріг $\geq 80\%$ — виконано)

HTML-звіт: <coverage/index.html>

6. Посилання

- Репозиторій: <https://github.com/oleksandrkononov1/bse-lr3-kononov>
 - CI: GitHub Actions автоматично запускає тести на кожен push та pull request
-

7. Висновки

У ході виконання лабораторної роботи було:

1. **Реалізовано програмний модуль** — 12 класів системи бібліографічних посилань на TypeScript, з нетривіальною логікою в методах `validate()`, `formatCitation()`, `generate()`.
2. **Спроектовано тест-кейси** з використанням технік EP та BVA:
 - Еквівалентне розбиття дозволило систематично покрити основні класи вхідних даних (валідні/невалідні DOI, ISBN, URL; різні стилі цитування; різна кількість авторів)
 - Аналіз граничних значень виявив критичні межі (мінімальний DOI, ISBN-10/13, пароль 7/8 символів, порожні вхідні дані)
3. **Написано 63 модульні тести** з використанням Vitest, кожен за патерном AAA з коментарем техніки.
4. **Досягнуто line coverage 96.11%**, що значно перевищує пороговий рівень 80%. Непокриті рядки — переважно делегуючі методи (`generateCitation()` у `Guest` та `RegisteredUser`) та інтерфейс `MetadataProvider` (не має runtime-коду).
5. **Налаштовано CI** через GitHub Actions для автоматичного запуску тестів.

Виявлені спостереження:

- Мокування `fetch` через `vi.stubGlobal()` дозволяє ізольовано тестувати провайдери без мережевих запитів
- Патерн AAA забезпечує чітку структуру тестів та їх читабельність
- Техніки EP та BVA доповнюють одна одну: EP визначає категорії, BVA — критичні точки всередині них